Project Design Phase-I

1. Proposed Solution Template

Metadata

- **Date:** June 19, 2026
- **Team ID:** ----------
- **Project Name:** AI-Enhanced Intrusion Detection System

Proposed Solution Table

S.No.	Parameter	Description
1.	**Problem Statement** *(Problem to be solved)*	Traditional firewalls rely on static, signature-based rules that fail to detect novel, zero-day network attacks or evolving malicious behaviors. This leaves critical systems vulnerable to high-velocity exploits like Denial of Service (DoS), unauthorized privilege escalation (U2R), and remote breaches (R2L) before signatures are updated.
2.	**Idea / Solution description**	An intelligent, machine learning-powered Network Intrusion Detection System (NIDS) that dynamically classifies incoming network packet metrics into five clear traffic states (Normal, DoS, Probe, R2L, U2R) using a high-accuracy Random Forest Classifier. It wraps this inference pipeline in a responsive web dashboard utilizing Python Flask and Bootstrap 5 for real-time monitoring and alert log management.
3.	**Novelty / Uniqueness**	Dual-purpose accessibility: It features a lightweight, high-performance in-memory alert logger for immediate local monitoring alongside an integrated standalone REST API endpoint (/api/predict). This design allows the application to serve simultaneously as an administrative visual interface and an automated backend microservice for larger security infrastructures.

S.No.	Parameter	Description
4.	**Social Impact / Customer Satisfaction**	Significantly reduces system administration overhead and democratizes robust network defense for smaller organizations or startups that cannot afford enterprise-grade SIEM solutions. Provides security personnel with low-latency visual summaries and automated logging to improve incident response speeds.
5.	**Business Model** *(Revenue Model)*	**Open-Core / Freemium Model:** • *Community Tier:* Free, open-source localized dashboard with manual log handling for small networks. • *Enterprise Tier (Subscription):* Paid commercial license unlocking persistent database monitoring, automated multi-node REST API sync, and live email/SMS webhook triggers upon high-confidence critical detections.
6.	**Scalability of the Solution**	Highly horizontally scalable at the API layer. Because the model uses an optimized tree-ensemble configuration and inputs are reduced to 10 key high-signal features, individual prediction requests require minimal memory and CPU cycles. The Flask microservice can be containerized using Docker and scaled behind a load balancer to ingest thousands of programmatic requests per second.